

LABORATORIO 03

El folio que llegó dos veces

Guía para leer mientras trabajas el cuaderno. Cada celda trae la pregunta que responde, la idea en el almacén, la sentencia explicada parte por parte, lo que sale en pantalla y cómo leerlo. Las salidas son las de la solución ejecutada del repositorio.

Introducción

1 · El tema

En este laboratorio trabajas con documentos tributarios de verdad, los que recibe la DGT, y con un problema que su área de datos tiene todos los días. El mismo documento llega dos veces. Vas a ver qué pasa si lo cargas sin pensar, cómo deshacer esa carga y cómo se carga bien, con una sola sentencia que decide documento por documento.

2 · El problema, en el almacén

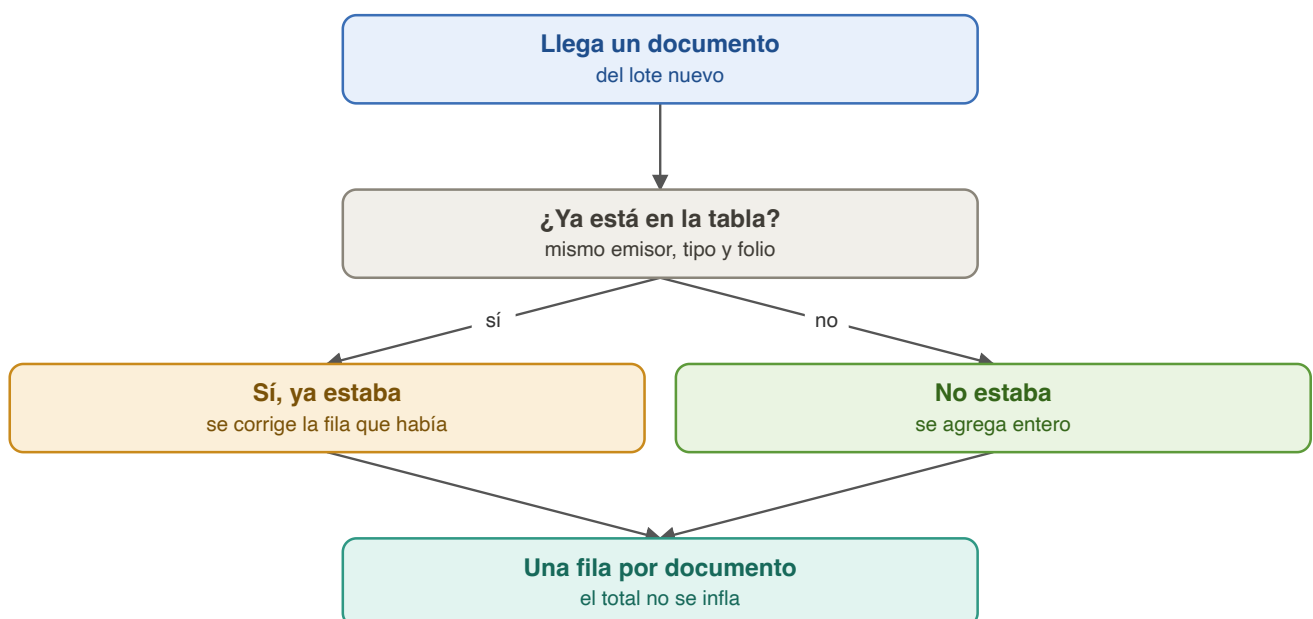
El proveedor entrega una factura, no recibe el acuse de recibo y la vuelve a mandar. Si el almacenero anota las dos, la libreta dice que compró el doble. Borrar toda la semana y volver a anotarla entera tampoco sirve, porque es mucho trabajo y se pierde el registro de lo que pasó. Lo que hace un buen almacenero es otra cosa. Busca la anotación que ya existe y la corrige, y si no la encuentra, la agrega.

3 · El problema en términos técnicos

El flujo de recepción de documentos trae reenvíos. Un contribuyente emite una factura, no ve la confirmación y la manda otra vez, así que el mismo folio llega dos veces. Si el lote se inserta tal cual, la tabla queda con dos filas para un solo documento y el total del mes sale inflado.

Las dos salidas habituales son malas. Insertar y arreglar después deja las cifras mal mientras tanto. Borrar el período completo y recargarlo cuesta horas y borra la historia. Hace falta una operación que mire cada documento que llega y decida sola si ya estaba o si es nuevo.

4 · El diagrama



Llega un documento (azul). Es cada fila del lote nuevo, un documento tributario que la DGT acaba de recibir.

¿Ya está en la tabla? (gris). Aquí se decide todo. Un documento se reconoce por tres datos juntos, quién lo emitió, de qué tipo es y su número de folio. Si en la tabla hay una fila con esos mismos tres valores, es el mismo

documento.

Sí, ya estaba (amarillo). Es un reenvío. No se agrega una fila más, se corrige la que ya había con lo que trae el reenvío.

No estaba (verde). Es un documento nuevo, y se agrega completo.

Una fila por documento (verde agua). Con esa decisión, cada documento queda una sola vez y el total del mes es el correcto.

5 · La solución

En el almacén, el almacenero toma cada factura que llega y la busca en la libreta por el nombre del proveedor, el tipo de papel y el número. Si la encuentra, corrige esa anotación. Si no la encuentra, la anota. Nunca hay dos anotaciones para la misma factura.

En Iceberg eso es la sentencia `MERGE INTO`, la fusión. Recibe la tabla, el lote que llega y la regla que dice cuándo dos filas son el mismo documento, y para cada documento hace una de dos cosas, corregir o agregar. Y si alguna vez una carga entra mal, la libreta guarda sus páginas y se puede volver a la anterior con `rollback_to_snapshot`, sin respaldos.

6 · Los pasos

- **Paso 0.** Creas la tabla `documentos_lab03` con el primer lote de recepción y miras qué trae.
- **Paso 1.** Llega el segundo lote y cuentas cuántos de sus documentos ya estaban.
- **Paso 2.** Insertas el lote tal cual, a propósito, y encuentras los folios repetidos.
- **Paso 3.** Deshaces esa carga volviendo a la página anterior de la libreta.
- **Paso 4.** Cargas el lote con `MERGE INTO` y compruebas que no queda ningún repetido.
- **Paso 5.** Corrige filas con `UPDATE` y borras filas con `DELETE`.
- **Cierre.** Las sentencias del laboratorio y los conteos de cada paso.

Para que el laboratorio salga como en esta guía, la tabla tiene que partir desde cero. La celda 0.2 la borra si existe, y si prefieres dejar todo limpio antes de empezar, corre el comando de abajo desde la carpeta del repositorio.

```
bash
bin/reiniciar-lab.sh 03
```

Paso 0 · Preparar la tabla

Celda 0.1 · Ponerte en tu espacio

La pregunta. ¿En qué espacio de trabajo van a quedar las tablas que crees?

Por qué ahora. Todo lo que crees en el laboratorio tiene que quedar en tu espacio, `mi_espacio`. Esta celda te pone ahí antes de crear nada.

En el almacén. Es pararte frente a tu propio mostrador antes de abrir la libreta.

La sentencia, parte por parte.

- `%%sql` es la primera línea de toda celda SQL del cuaderno. Le dice al cuaderno que lo que sigue es SQL y no Python.
- `-- Celda 0.1` es un comentario. Todo lo que va después de dos guiones lo ignora el motor, y sirve para saber qué celda es.
- `USE mi_espacio` deja a `mi_espacio` como espacio de trabajo. Desde aquí, una tabla nombrada sin espacio adelante se busca y se crea en `mi_espacio`.

```
sql
%%sql
-- Celda 0.1
USE mi_espacio
```

Lo que sale en pantalla.

```
text
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/09/25 18:22:14 WARN NativeCodeLoader: Unable to load native-hadoop library for your
platform... using builtin-java classes where applicable
Listo. La sentencia se ejecutó.
```

Cómo se lee. La sentencia funcionó y ya estás en tu espacio. `USE` no devuelve filas, por eso el cuaderno solo avisa que se ejecutó.

Las tres primeras líneas no son errores. Salen solo en la primera celda que usa Spark, cuando arranca.

- `Setting default log level to "WARN"` dice que Spark va a mostrar solo los avisos y los errores, no todo lo que hace.
- `To adjust logging level use sc.setLogLevel(newLevel)` explica cómo cambiar eso. `sc` es el contexto de Spark y `setLogLevel` la función que cambia el nivel. No hace falta tocarlo. SparkR es la versión de Spark para el lenguaje R, que aquí no se usa.
- `WARN NativeCodeLoader: Unable to load native-hadoop library` dice que Spark no encontró una biblioteca de Hadoop escrita para este sistema operativo y que usa la versión en Java. Funciona igual. `26/09/25 18:22:14` es la fecha y la hora del aviso, año 2026, mes 09, día 25, en hora UTC.

Celda 0.2 · Partir de cero

La pregunta. ¿Qué pasa si ya existe una tabla con ese nombre de una vez anterior?

Por qué ahora. Si corres el laboratorio por segunda vez, la tabla ya existe y la creación de la celda siguiente fallaría. Esta celda la borra antes.

En el almacén. Es sacar del mostrador la libreta vieja de este ejercicio, si quedó alguna, para empezar con una nueva.

La sentencia, parte por parte.

- `DROP TABLE` borra la tabla.
- `IF EXISTS` hace que no falle si la tabla no existe. Si no está, la sentencia no hace nada.
- `documentos_lab03` es el nombre de la tabla. Como no lleva espacio adelante, se busca en `mi_espacio`.

```
sql
%%sql
-- Celda 0.2
DROP TABLE IF EXISTS documentos_lab03
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Se ejecutó sin problema, existiera o no la tabla.

Celda 0.3 · Crear la tabla con el primer lote

La pregunta. ¿Cómo se crea una tabla ya cargada, sin escribir sus columnas una por una?

Por qué ahora. Los documentos de verdad no se teclean. Se cargan desde lo que ya existe. Esta celda crea la tabla y la llena con el primer lote de recepción en una sola sentencia.

En el almacén. Es abrir una libreta nueva y copiar en ella, de una vez, todas las facturas que llegaron el primer día.

La sentencia, parte por parte.

- `CREATE TABLE documentos_lab03` crea la tabla con ese nombre, en `mi_espacio`.
- `USING iceberg` hace que sea una tabla Iceberg, con libreta, páginas y todo lo demás.
- `AS SELECT * FROM curso.recepcion_lote1` le da las columnas y los tipos que devuelve esa consulta, y la carga con sus filas. `curso.recepcion_lote1` es el primer lote de documentos recibidos, que ya viene en el espacio `curso`, el de las tablas de referencia. Tú solo lo lees.

```
sql
%%sql
-- Celda 0.3
CREATE TABLE documentos_lab03 USING iceberg AS
SELECT * FROM curso.recepcion_lote1
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La tabla quedó creada y cargada. Esta carga deja la primera página de la libreta.

Celda 0.4 · Contar los documentos

La pregunta. ¿Cuántos documentos trajo el primer lote?

Por qué ahora. Todo el laboratorio se sigue contando. Este es el número de partida.

En el almacén. Es contar las facturas anotadas en la libreta nueva.

La sentencia, parte por parte.

- `SELECT count(*)` cuenta las filas. El asterisco significa todas, sin fijarse en ninguna columna.
- `AS documentos` le pone nombre a la columna del resultado.
- `FROM documentos_lab03` es tu tabla.

```
sql
%%sql
-- Celda 0.4
SELECT count(*) AS documentos FROM documentos_lab03
```

Lo que sale en pantalla.

```
text
documentos
30
1 fila.
```

Cómo se lee. Treinta documentos. Es el primer lote completo.

Celda 0.5 · Mirar algunos documentos

La pregunta. ¿Qué trae cada documento?

Por qué ahora. Antes de cargar el lote siguiente conviene saber qué columnas tiene un documento y cuáles lo identifican.

En el almacén. Es leer las primeras anotaciones de la libreta para saber cómo están escritas.

La sentencia, parte por parte.

- `SELECT rut_emisor, tipo_dte, folio, fecha_recepcion, monto_total, estado_sii` elige seis columnas. `rut_emisor` es el RUT de quien emitió el documento, `tipo_dte` el tipo de documento, `folio` su número, `fecha_recepcion` cuándo lo recibió la DGT, `monto_total` el monto y `estado_sii` en qué estado quedó.
- `FROM documentos_lab03` es tu tabla.
- `ORDER BY rut_emisor, tipo_dte, folio` ordena por emisor, después por tipo y después por folio, para que el resultado salga siempre igual.
- `LIMIT 5` se queda con las cinco primeras filas.

```
sql
%%sql
-- Celda 0.5
SELECT rut_emisor, tipo_dte, folio, fecha_recepcion, monto_total, estado_sii
FROM documentos_lab03
ORDER BY rut_emisor, tipo_dte, folio
LIMIT 5
```

Lo que sale en pantalla.

```
text
rut_emisor  tipo_dte  folio  fecha_recepcion  monto_total  estado_sii
76030854-4  39        864    2026-06-06 17:23:16  29857.10     ACEPTADO
76034637-3  39        860    2026-06-08 03:17:22  70864.50     ACEPTADO
76057484-8  33        226    2026-06-12 00:29:09  6972860.93   ACEPTADO
76133016-0  61        12     2026-06-10 03:23:20  -895572.58   ACEPTADO
76252637-9  33        972    2026-06-07 08:21:09  17190434.17  ACEPTADO
5 filas.
```

Cómo se lee. Cinco documentos, uno por fila.

- `tipo_dte` es un código. 33 es factura afecta, 34 factura exenta, 39 boleta, 52 guía de despacho, 56 nota de débito y 61 nota de crédito.
- La cuarta fila es una nota de crédito, tipo 61, y por eso su monto es negativo, `-895572.58`. Una nota de crédito descuenta.
- `fecha_recepcion` trae fecha y hora, por ejemplo `2026-06-12 00:29:09`. Es un dato del documento, no una hora del sistema.
- `ACEPTADO` es el estado con que quedó cada documento.

El documento del emisor `76057484-8`, tipo 33, folio 226, es el que vas a seguir de cerca en los pasos 2 y 4.

Paso 1 · Llega el lote nuevo

Celda 1.1 · Contar el segundo lote

La pregunta. ¿Cuántos documentos trae el lote del día siguiente?

Por qué ahora. Antes de cargar un lote se mira qué trae. El segundo lote está en `curso.recepcion_lote2`.

En el almacén. Es contar las facturas que llegaron hoy, antes de anotarlas.

La sentencia, parte por parte.

- `SELECT count(*)` cuenta las filas.
- `AS documentos` le pone nombre a la columna.
- `FROM curso.recepcion_lote2` es el segundo lote, en el espacio `curso`. Se nombra con su espacio adelante porque no está en el tuyo.

```
sql
%%sql
-- Celda 1.1
SELECT count(*) AS documentos FROM curso.recepcion_lote2
```

Lo que sale en pantalla.

```
text
documentos
20
1 fila.
```

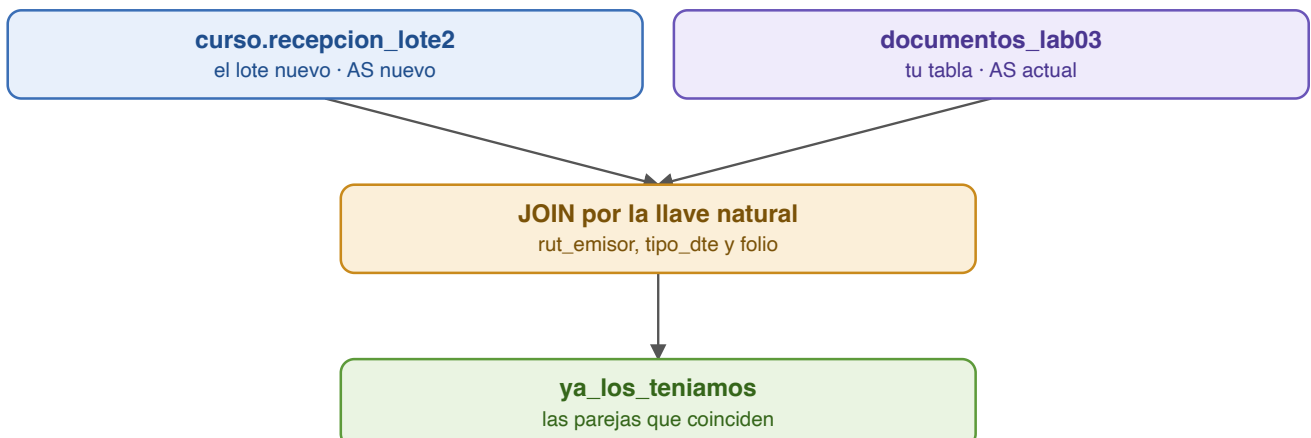
Cómo se lee. Veinte documentos llegan en el segundo lote.

Celda 1.2 · Cuántos ya los tenías

La pregunta. ¿Cuántos de esos veinte ya están en tu tabla?

Por qué ahora. Si algunos ya están, son reenvíos, y cargarlos sin cuidado los va a duplicar. Hay que saberlo antes.

En el almacén. Es tomar cada factura que llegó hoy y buscarla en la libreta por proveedor, tipo de papel y número.



curso.recepcion_lote2 (azul). El lote nuevo, con el apodo **nuevo**.

documentos_lab03 (morado). Tu tabla, con el apodo **actual**.

JOIN por la llave natural (amarillo). Junta cada documento del lote con el de la tabla que tenga el mismo emisor, el mismo tipo y el mismo folio. Esas tres columnas juntas son la llave natural del documento. Dos filas con los mismos tres valores son el mismo documento.

ya_los_teniamos (verde). Cuántas parejas se formaron, es decir, cuántos documentos del lote ya estaban en la tabla.

La sentencia, parte por parte.

- **SELECT count(*) AS ya_los_teniamos** cuenta las parejas y le pone ese nombre a la columna.
- **FROM curso.recepcion_lote2 AS nuevo** es el lote nuevo. **AS nuevo** le pone un apodo para no repetir el nombre largo.
- **JOIN documentos_lab03 AS actual** junta el lote con tu tabla, que se llama **actual** en esta consulta. Un **JOIN** solo deja las filas que encuentran pareja.
- **ON actual.rut_emisor = nuevo.rut_emisor** es la primera condición de la pareja, el mismo emisor.
- **AND actual.tipo_dte = nuevo.tipo_dte** agrega el mismo tipo de documento.
- **AND actual.folio = nuevo.folio** agrega el mismo folio. Las tres condiciones tienen que cumplirse a la vez.

```
sql
%%sql
-- Celda 1.2
SELECT count(*) AS ya_los_teniamos
FROM curso.recepcion_lote2 AS nuevo
JOIN documentos_lab03 AS actual
  ON actual.rut_emisor = nuevo.rut_emisor
  AND actual.tipo_dte  = nuevo.tipo_dte
  AND actual.folio     = nuevo.folio
```

Lo que sale en pantalla.

```
text
ya_los_teniamos
12
1 fila.
```

Cómo se lee. Doce documentos del lote nuevo ya estaban en la tabla. Son los reenvíos. Los otros ocho son nuevos.

Paso 2 · El error que hay que ver

Celda 2.1 · Insertar el lote tal cual

La pregunta. ¿Qué pasa si cargas el lote nuevo sin fijarte en los reenvíos?

Por qué ahora. Es la sentencia que cualquiera escribiría sin pensar. Se corre a propósito para ver el daño en tu propia tabla, y en el paso 3 se deshace.

En el almacén. Es anotar todas las facturas de hoy al final de la libreta, sin mirar si alguna ya estaba anotada.

La sentencia, parte por parte.

- `INSERT INTO documentos_lab03` agrega filas a tu tabla.
- `SELECT * FROM curso.recepcion_lote2` son las filas que se agregan, todas las del segundo lote con todas sus columnas.

```
sql
%%sql
-- Celda 2.1
INSERT INTO documentos_lab03 SELECT * FROM curso.recepcion_lote2
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Entró sin quejarse. La tabla no tiene ninguna regla que impida repetir un documento, y esta carga deja una segunda página en la libreta.

Celda 2.2 · Contar otra vez

La pregunta. ¿Cuántos documentos quedaron?

Por qué ahora. Es la forma más rápida de ver si la carga hizo lo que se esperaba.

En el almacén. Es contar de nuevo las anotaciones de la libreta.

La sentencia, parte por parte.

- `SELECT count(*) AS documentos` cuenta las filas y nombra la columna.
- `FROM documentos_lab03` es tu tabla.

```
sql
%%sql
-- Celda 2.2
SELECT count(*) AS documentos FROM documentos_lab03
```

Lo que sale en pantalla.

```
text
documentos
50
1 fila.
```

Cómo se lee. Cincuenta, treinta más veinte. La suma está bien hecha y el resultado está mal, porque doce de esos documentos están dos veces.

Celda 2.3 · Encontrar los repetidos

La pregunta. ¿Cuáles son los documentos que quedaron dos veces?

Por qué ahora. Saber que hay repetidos no basta. Hay que poder nombrarlos.

En el almacén. Es ordenar las anotaciones por proveedor, tipo y número, y marcar las que aparecen más de una vez.

La sentencia, parte por parte.

- `SELECT rut_emisor, tipo_dte, folio, count(*) AS veces` muestra la llave natural de cada documento y cuántas filas tiene.
- `FROM documentos_lab03` es tu tabla.
- `GROUP BY rut_emisor, tipo_dte, folio` junta en un grupo todas las filas con la misma llave. `count(*)` cuenta las filas de cada grupo.
- `HAVING count(*) > 1` se queda solo con los grupos de más de una fila. `HAVING` filtra grupos, igual que `WHERE` filtra filas.
- `ORDER BY rut_emisor, tipo_dte, folio` ordena el resultado para que salga siempre igual.

```
sql
%%sql
-- Celda 2.3
SELECT rut_emisor, tipo_dte, folio, count(*) AS veces
FROM documentos_lab03
GROUP BY rut_emisor, tipo_dte, folio
HAVING count(*) > 1
ORDER BY rut_emisor, tipo_dte, folio
```

Lo que sale en pantalla.

```
text
rut_emisor  tipo_dte  folio  veces
76057484-8  33        226    2
76133016-0  61        12     2
76705019-4  34        149    2
76899472-2  61        28     2
76913181-7  39        778    2
77612645-4  61        100    2
77781727-2  39        15     2
78086012-K  39        223    2
78914770-1  61        10     2
79068554-7  33        52     2
79324470-3  33        16     2
79961054-K  39        889    2
12 filas.
```

Cómo se lee. Doce documentos, cada uno con `veces` igual a 2. Son exactamente los doce reenvíos que contó la celda 1.2. El primero de la lista es el del emisor `76057484-8`, tipo 33, folio 226.

Celda 2.4 · Mirar un repetido de cerca

La pregunta. ¿En qué se diferencian las dos filas de un mismo documento?

Por qué ahora. Si las dos filas fueran idénticas, bastaría con borrar una. Hay que ver qué cambia entre la original y el reenvío.

En el almacén. Es poner lado a lado las dos anotaciones de la misma factura.

La sentencia, parte por parte.

- `SELECT rut_emisor, tipo_dte, folio, fecha_recepcion, monto_total, estado_sii` elige las columnas para comparar.
- `FROM documentos_lab03` es tu tabla.
- `WHERE rut_emisor = '76057484-8' AND tipo_dte = 33 AND folio = 226` se queda solo con ese documento. El RUT va entre comillas porque es texto, y el tipo y el folio van sin comillas porque son números.
- `ORDER BY fecha_recepcion` ordena por la hora de recepción, para que la original salga primero y el reenvío después.

```
sql
%%sql
-- Celda 2.4
SELECT rut_emisor, tipo_dte, folio, fecha_recepcion, monto_total, estado_sii
FROM documentos_lab03
WHERE rut_emisor = '76057484-8' AND tipo_dte = 33 AND folio = 226
ORDER BY fecha_recepcion
```

Lo que sale en pantalla.

```
text
rut_emisor  tipo_dte  folio  fecha_recepcion  monto_total  estado_sii
76057484-8  33        226    2026-06-12 00:29:09  6972860.93  ACEPTADO
76057484-8  33        226    2026-06-13 15:41:09  6972860.93  ACEPTADO
2 filas.
```

Cómo se lee. El mismo documento dos veces. Lo único distinto es `fecha_recepcion`. La primera llegó el `2026-06-12 00:29:09` y el reenvío el `2026-06-13 15:41:09`. El monto, `6972860.93`, está contado dos veces en cualquier suma.

Paso 3 · Deshacer

Celda 3.1 · Mirar las páginas

La pregunta. ¿Qué páginas tiene la libreta ahora?

Por qué ahora. Para deshacer la carga mala sin borrar filas a mano, hay que volver a la página de antes de esa carga. Primero hay que ver cuál es.

En el almacén. Es mirar la lista de páginas de la libreta, cada una con su hora.

La sentencia, parte por parte.

- `SELECT snapshot_id, committed_at, operation` pide tres columnas. `snapshot_id` es el número de la página, `committed_at` la hora en que se colgó, en UTC, y `operation` qué se hizo en ella.
- `FROM mi_espacio.documentos_lab03.snapshots` lee la vista de sistema `.snapshots`, la lista de páginas. El nombre va con tres partes, el espacio, la tabla y la vista, aunque ya hayas hecho `USE`. Con solo dos partes, SQL creería que `documentos_lab03` es un espacio.
- `ORDER BY committed_at` ordena de la página más vieja a la más nueva.

```
sql
%%sql
-- Celda 3.1
SELECT snapshot_id, committed_at, operation
FROM mi_espacio.documentos_lab03.snapshots
ORDER BY committed_at
```

Lo que sale en pantalla.

```
text
snapshot_id      committed_at      operation
7877079783851262128  2026-09-25 18:22:19.586  append
8377222337610389612  2026-09-25 18:22:21.583  append
2 filas.
```

Cómo se lee. Dos páginas, las dos `append`, que significa que se agregaron filas.

- La primera, `7877079783851262128`, es la carga inicial de la celda 0.3, a las 18:22:19 UTC, las 15:22 en Chile.
- La segunda, `8377222337610389612`, es el `INSERT` de la celda 2.1, dos segundos después.

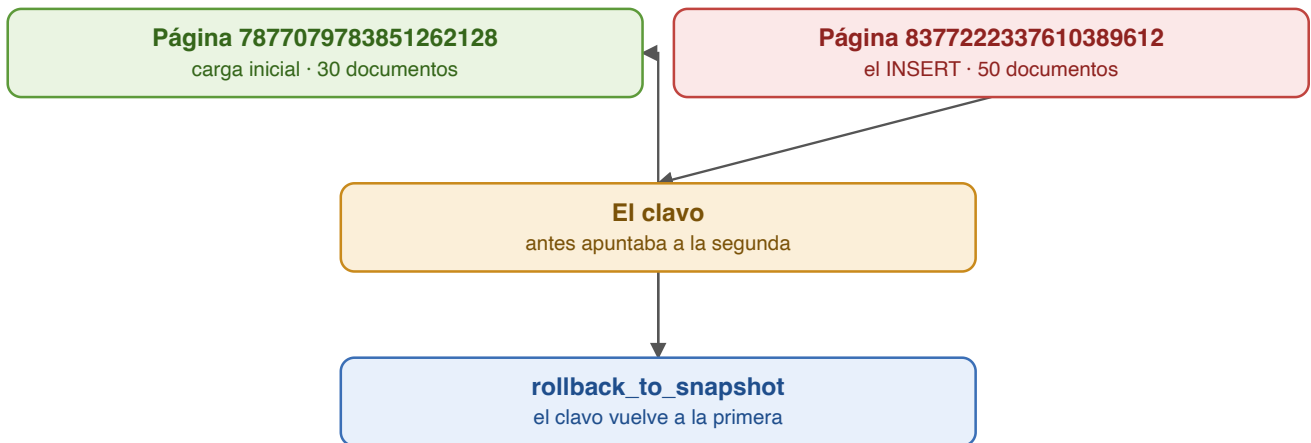
En tu pantalla los números y las horas van a ser otros, porque cada tabla genera los suyos.

Celda 3.2 · Volver a la primera página

La pregunta. ¿Cómo se deshace una carga entera sin borrar filas a mano?

Por qué ahora. La tabla tiene doce documentos duplicados. En vez de buscarlos y borrarlos, se declara vigente la página de antes de la carga mala.

En el almacén. Es volver a colgar en el clavo la página de ayer. La de hoy no se rompe, simplemente deja de ser la que manda.



Página 7877079783851262128 (verde). La página de la carga inicial, con los treinta documentos.

Página 8377222337610389612 (rojo). La página del `INSERT` equivocado, con los cincuenta.

El clavo (amarillo). El catálogo, que dice cuál página es la vigente. Antes del rollback apuntaba a la segunda.

rollback_to_snapshot (azul). El procedimiento que mueve el clavo a otra página. Después de correrlo, la vigente vuelve a ser la primera.

La sentencia, parte por parte.

- `CALL` ejecuta un procedimiento, que es una operación del sistema, no una consulta.
- `spark_catalog.system.rollback_to_snapshot` es el procedimiento. `spark_catalog` es el nombre del catálogo, `system` el grupo de procedimientos de Iceberg y `rollback_to_snapshot` el que vuelve a una página.
- `'mi_espacio.documentos_lab03'` es la tabla, entre comillas y con su espacio adelante.
- `<TU_SNAPSHOT_ID>` es un marcador. Donde está, escribes sin comillas el `snapshot_id` de la primera fila de la celda 3.1. En la solución ese número fue `7877079783851262128`.

```
sql
%%sql
-- Celda 3.2
CALL spark_catalog.system.rollback_to_snapshot('mi_espacio.documentos_lab03', <TU_SNAPSHOT_ID>)
```

Lo que sale en pantalla.

```
text
previous_snapshot_id  current_snapshot_id
8377222337610389612  7877079783851262128
1 fila.
```

Cómo se lee. El clavo se movió. `previous_snapshot_id` es la página que mandaba antes, `8377222337610389612`, y `current_snapshot_id` la que manda ahora, `7877079783851262128`, la de la carga inicial.

Celda 3.3 · Comprobar

La pregunta. ¿Volvió la tabla a sus treinta documentos?

Por qué ahora. Después de mover el clavo hay que confirmar que la tabla dice lo que se esperaba.

En el almacén. Es contar de nuevo las anotaciones de la página que quedó colgada.

La sentencia, parte por parte.

- `SELECT count(*) AS documentos` cuenta las filas.
- `FROM documentos_lab03` es tu tabla, tal como está ahora.

```
sql
%%sql
-- Celda 3.3
SELECT count(*) AS documentos FROM documentos_lab03
```

Lo que sale en pantalla.

```
text
documentos
30
1 fila.
```

Cómo se lee. Treinta otra vez. La carga mala quedó deshecha con una sentencia, sin restaurar ningún respaldo.

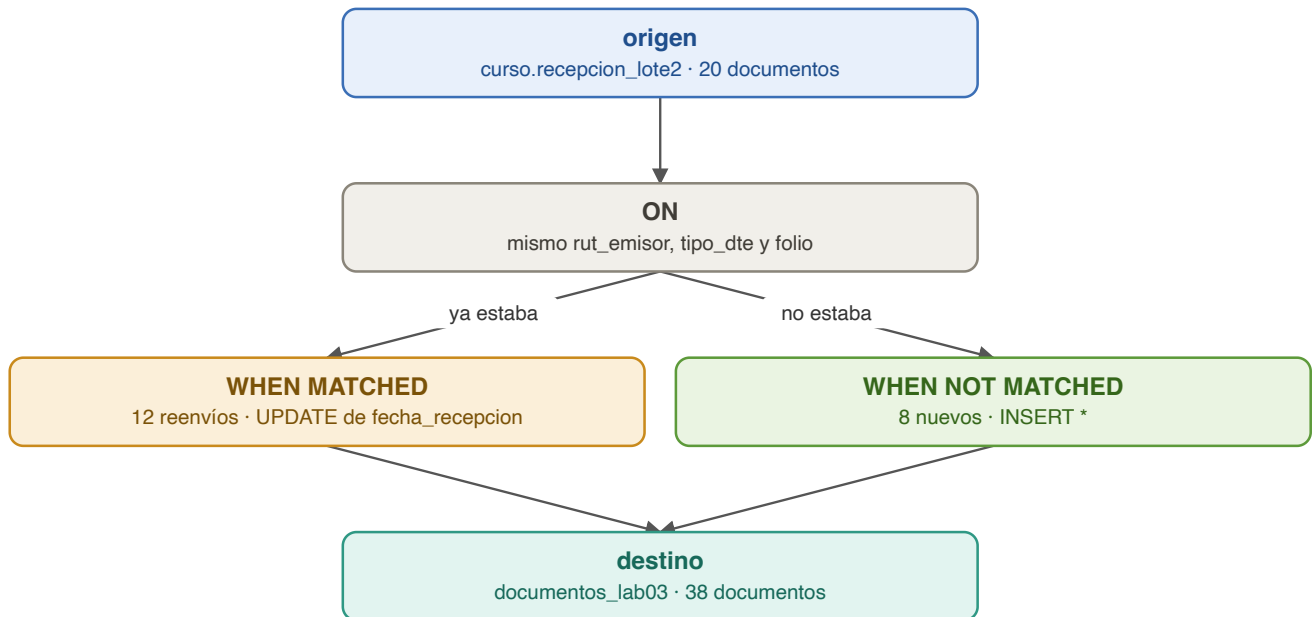
Paso 4 · La fusión

Celda 4.1 · Cargar con MERGE INTO

La pregunta. ¿Cómo se carga el lote sin duplicar los reenvíos?

Por qué ahora. La tabla está como antes del error. Ahora se carga el mismo lote de la forma correcta.

En el almacén. El almacenero toma cada factura de hoy, la busca en la libreta y decide. Si ya estaba, corrige la anotación con la fecha nueva. Si no estaba, la anota.



origen (azul). El lote que llega, `curso.recepcion_lote2`, con sus veinte documentos.

ON (gris). La regla que dice cuándo un documento del lote y uno de la tabla son el mismo, la llave natural de tres columnas.

WHEN MATCHED (amarillo). Lo que se hace con los documentos que ya estaban, los doce reenvíos. Se corrige su fecha de recepción con la del reenvío.

WHEN NOT MATCHED (verde). Lo que se hace con los que no estaban, los ocho nuevos. Se agregan enteros.

destino (verde agua). Tu tabla después de la fusión, con treinta más ocho documentos.

La sentencia, parte por parte.

- `MERGE INTO documentos_lab03 AS destino` es la tabla que se va a modificar, con el apodo `destino`.
- `USING curso.recepcion_lote2 AS origen` es de dónde vienen las filas que llegan, con el apodo `origen`.
- `ON destino.rut_emisor = origen.rut_emisor` es la primera parte de la llave, el mismo emisor.
- `AND destino.tipo_dte = origen.tipo_dte` agrega el mismo tipo.
- `AND destino.folio = origen.folio` agrega el mismo folio.
- `WHEN MATCHED THEN UPDATE SET destino.fecha_recepcion = origen.fecha_recepcion` dice qué hacer si el documento ya estaba. Se corrige la fecha de recepción de la tabla con la del reenvío, y nada más.
- `WHEN NOT MATCHED THEN INSERT *` dice qué hacer si no estaba. Se agrega con todas sus columnas. El asterisco significa todas.

```

sql
%%sql
-- Celda 4.1
MERGE INTO documentos_lab03 AS destino
USING curso.recepcion_lote2 AS origen
  ON destino.rut_emisor = origen.rut_emisor
  AND destino.tipo_dte  = origen.tipo_dte
  AND destino.folio     = origen.folio
WHEN MATCHED THEN UPDATE SET destino.fecha_recepcion = origen.fecha_recepcion
WHEN NOT MATCHED THEN INSERT *

```

Lo que sale en pantalla.

```

text
Listo. La sentencia se ejecutó.

```

Cómo se lee. La fusión se hizo en una sola sentencia. La decisión de cada documento la tomó el motor, no tú, y quedó anotada como una página nueva de la libreta.

Celda 4.2 · Contar después de la fusión

La pregunta. ¿Cuántos documentos quedaron esta vez?

Por qué ahora. Es la primera prueba de que la fusión no duplicó nada.

En el almacén. Es contar las anotaciones después de revisar las facturas de hoy.

La sentencia, parte por parte.

- `SELECT count(*) AS documentos` cuenta las filas.
- `FROM documentos_lab03` es tu tabla.

```

sql
%%sql
-- Celda 4.2
SELECT count(*) AS documentos FROM documentos_lab03

```

Lo que sale en pantalla.

```

text
documentos
38
1 fila.

```

Cómo se lee. Treinta y ocho, no cincuenta. Los doce reenvíos no agregaron filas, corrigieron las que ya estaban. Los ocho nuevos sí entraron. Treinta más ocho da treinta y ocho.

Celda 4.3 · Volver al documento repetido

La pregunta. ¿Cómo quedó el documento que antes estaba dos veces?

Por qué ahora. El conteo cuadra, pero conviene ver con los ojos un caso concreto.

En el almacén. Es buscar la factura que llegó dos veces y ver cuántas anotaciones tiene ahora.

La sentencia, parte por parte.

- `SELECT rut_emisor, tipo_dte, folio, fecha_recepcion, monto_total, estado_sii` elige las mismas columnas de la celda 2.4.
- `FROM documentos_lab03` es tu tabla.
- `WHERE rut_emisor = '76057484-8' AND tipo_dte = 33 AND folio = 226` se queda con ese documento.

```
sql
%%sql
-- Celda 4.3
SELECT rut_emisor, tipo_dte, folio, fecha_recepcion, monto_total, estado_sii
FROM documentos_lab03
WHERE rut_emisor = '76057484-8' AND tipo_dte = 33 AND folio = 226
```

Lo que sale en pantalla.

```
text
rut_emisor  tipo_dte  folio  fecha_recepcion      monto_total  estado_sii
76057484-8  33         226    2026-06-13 15:41:09  6972860.93  ACEPTADO
1 fila.
```

Cómo se lee. Una sola fila, con la fecha de recepción del reenvío, `2026-06-13 15:41:09`, la más tardía de las dos. El dato quedó actualizado, no duplicado.

Celda 4.4 · Buscar repetidos en toda la tabla

La pregunta. ¿Queda algún documento repetido?

Por qué ahora. Un caso bien no prueba que todos estén bien. Esta celda revisa la tabla completa.

En el almacén. Es recorrer toda la libreta buscando facturas anotadas dos veces.

La sentencia, parte por parte.

- `SELECT rut_emisor, tipo_dte, folio FROM documentos_lab03 GROUP BY rut_emisor, tipo_dte, folio HAVING count(*) > 1` es la consulta de adentro, entre paréntesis. Es la misma búsqueda de la celda 2.3, y devuelve un documento por fila, uno por cada repetido.
- `SELECT count(*) AS folios_repetidos FROM ( ... )` cuenta cuántas filas devolvió la consulta de adentro. Una consulta dentro de otra se llama subconsulta.

```
sql
%%sql
-- Celda 4.4
SELECT count(*) AS folios_repetidos FROM (
  SELECT rut_emisor, tipo_dte, folio
  FROM documentos_lab03
  GROUP BY rut_emisor, tipo_dte, folio
  HAVING count(*) > 1
)
```

Lo que sale en pantalla.

```
text
folios_repetidos
0
1 fila.
```

Cómo se lee. Cero. No queda ningún documento repetido en la tabla.

Paso 5 · Corregir y borrar

Celda 5.1 · Marcar las notas de crédito

La pregunta. ¿Cómo se corrigen todas las filas que cumplen una condición?

Por qué ahora. Además de cargar sin duplicar, el trabajo diario pide corregir y borrar. En Iceberg se hace con las mismas sentencias de cualquier base de datos.

En el almacén. Es pasar por la libreta y poner la marca de revisado a todas las notas de crédito.

La sentencia, parte por parte.

- `UPDATE documentos_lab03` dice qué tabla se corrige.
- `SET estado_sii = 'REVISADO'` es el cambio, poner `REVISADO` en la columna del estado.
- `WHERE tipo_dte = 61` dice a qué filas, solo a las notas de crédito.

```
sql
%%sql
-- Celda 5.1
UPDATE documentos_lab03 SET estado_sii = 'REVISADO' WHERE tipo_dte = 61
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La corrección se hizo y quedó como una página nueva de la libreta.

Celda 5.2 · Contar por estado

La pregunta. ¿A cuántos documentos afectó la corrección?

Por qué ahora. `UPDATE` no dice cuántas filas tocó. Hay que contarlo.

En el almacén. Es separar las anotaciones según su marca y contar cada montón.

La sentencia, parte por parte.

- `SELECT estado_sii, count(*) AS documentos` muestra cada estado y cuántos documentos tiene.
- `FROM documentos_lab03` es tu tabla.
- `GROUP BY estado_sii` junta las filas por estado.
- `ORDER BY estado_sii` ordena alfabéticamente.

```
sql
%%sql
-- Celda 5.2
SELECT estado_sii, count(*) AS documentos
FROM documentos_lab03
GROUP BY estado_sii
ORDER BY estado_sii
```

Lo que sale en pantalla.

```
text
estado_sii      documentos
ACEPTADO        31
ACEPTADO_CON_REPAROS  2
REVISADO        5
3 filas.
```

Cómo se lee. Cinco documentos quedaron en `REVISADO`, que son las cinco notas de crédito. Treinta y uno siguen `ACEPTADO` y dos están en `ACEPTADO_CON_REPAROS`, un estado que ya traían algunos documentos del lote. Treinta y uno más dos más cinco da treinta y ocho.

Celda 5.3 · Borrar los montos negativos

La pregunta. ¿Cómo se borran las filas que cumplen una condición?

Por qué ahora. Es la otra operación del día a día. En este lote, los documentos de monto negativo son las notas de crédito.

En el almacén. Es sacar de la libreta todas las anotaciones con monto negativo.

La sentencia, parte por parte.

- `DELETE FROM documentos_lab03` dice de qué tabla se borra.
- `WHERE monto_total < 0` dice qué filas, las de monto menor que cero.

```
sql
%%sql
-- Celda 5.3
DELETE FROM documentos_lab03 WHERE monto_total < 0
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. El borrado se hizo. Como toda escritura, dejó una página nueva, así que las filas borradas siguen visibles en las páginas anteriores.

Celda 5.4 · El recuento final

La pregunta. ¿Cuántos documentos quedaron al final?

Por qué ahora. Cierra el laboratorio con el número que tiene que cuadrar.

En el almacén. Es el último conteo de la libreta.

La sentencia, parte por parte.

- `SELECT count(*) AS documentos` cuenta las filas.
- `FROM documentos_lab03` es tu tabla.

```
sql
%%sql
-- Celda 5.4
SELECT count(*) AS documentos FROM documentos_lab03
```

Lo que sale en pantalla.

```
text
documentos
33
1 fila.
```

Cómo se lee. Treinta y tres. Eran treinta y ocho y se borraron las cinco notas de crédito.

Preguntas frecuentes

¿Por qué la tabla dejó entrar documentos repetidos?

Porque una tabla Iceberg no tiene llave primaria que lo impida. Aceptar o rechazar un documento repetido es trabajo de quien carga. Por eso existe `MERGE INTO`, que revisa la llave natural antes de escribir.

¿Qué es la llave natural?

Las columnas que, juntas, identifican un documento en el mundo real. Aquí son el RUT del emisor, el tipo de documento y el folio. El folio solo no basta, porque dos emisores distintos pueden tener el mismo número de folio, y un mismo emisor numera por separado sus facturas y sus boletas.

¿Qué pasa si el reenvío trae un monto distinto?

Con la sentencia de la celda 4.1, nada, porque el `WHEN MATCHED` solo corrige la fecha de recepción. Si quisieras que el reenvío también corrija el monto o el estado, se agregan esas columnas al `SET`, separadas por comas.

¿El rollback borró la página del INSERT equivocado?

No. La página sigue en la libreta, solo que ya no es la vigente. El rollback mueve el clavo, no borra papelitos. Si la listas en `.snapshots` después, sigue apareciendo.

¿De dónde saco el número que va en `<TU_SNAPSHOT_ID>` ?

De la salida de la celda 3.1, la columna `snapshot_id` de la primera fila, la de la carga inicial. Se escribe sin comillas. El de la solución fue `7877079783851262128`, pero el tuyo es otro, porque cada tabla genera sus propios números.

¿MERGE INTO reescribe toda la tabla?

No toda. Reescribe los papelitos que contienen documentos que se corrigieron y agrega papelitos para los nuevos. Queda una sola página nueva en la libreta, igual que con cualquier otra escritura.

¿Por qué el UPDATE y el DELETE no dicen cuántas filas cambiaron?

Porque el cuaderno solo avisa que la sentencia se ejecutó. Para saber cuántas filas tocó, se cuenta después, como en las celdas 5.2 y 5.4.

¿Qué pasa si corro el cuaderno dos veces?

La celda 0.2 borra la tabla y todo parte de cero, así que los conteos salen iguales. Solo cambian los números de las páginas y las horas. Si prefieres partir limpio antes, corre `bin/reiniciar-lab.sh 03`.

Cierre

La pregunta. ¿Qué te llevas de este laboratorio?

Una forma de cargar lotes que traen reenvíos sin duplicar nada, y una forma de deshacer una carga mala sin tocar respaldos.

En el almacén. El almacenero ya no anota dos veces la misma factura. Busca, corrige lo que ya estaba y agrega lo nuevo. Y cuando anota algo mal, vuelve a colgar la página de ayer.



INSERT sin mirar (rojo). Cargar el lote tal cual dejó cincuenta documentos, doce de ellos repetidos.

rollback (azul). Volver a la página anterior deshizo esa carga en una sentencia.

MERGE INTO (verde). La fusión dejó treinta y ocho documentos y ningún repetido, decidiendo documento por documento.

Los números de la solución ejecutada.

Momento	Documentos	Qué pasó
Primer lote	30	la carga inicial
Después del INSERT	50	doce reenvíos duplicados
Después del rollback	30	la carga mala deshecha
Después del MERGE INTO	38	doce corregidos y ocho nuevos
Después del DELETE	33	cinco notas de crédito borradas

Lo que necesitas	La sentencia
Cargar una tabla desde una consulta	<code>CREATE TABLE ... USING iceberg AS SELECT ...</code>
Agregar filas	<code>INSERT INTO ... SELECT ...</code>
Cargar sin duplicar	<code>MERGE INTO ... USING ... ON ... WHEN MATCHED ... WHEN NOT MATCHED ...</code>
Corregir filas	<code>UPDATE ... SET ... WHERE ...</code>
Borrar filas	<code>DELETE FROM ... WHERE ...</code>
Deshacer una carga entera	<code>CALL ... rollback_to_snapshot(...)</code>

Cargar sin duplicar ya no significa borrar el período y reconstruirlo. Es una sentencia que decide documento por documento, y la libreta guarda cada paso por si hay que volver atrás.